

---

# Solução de Recuperação de Crédito PJ - Itaú

Robert Takeshi Naito Avano  
05/03/2025

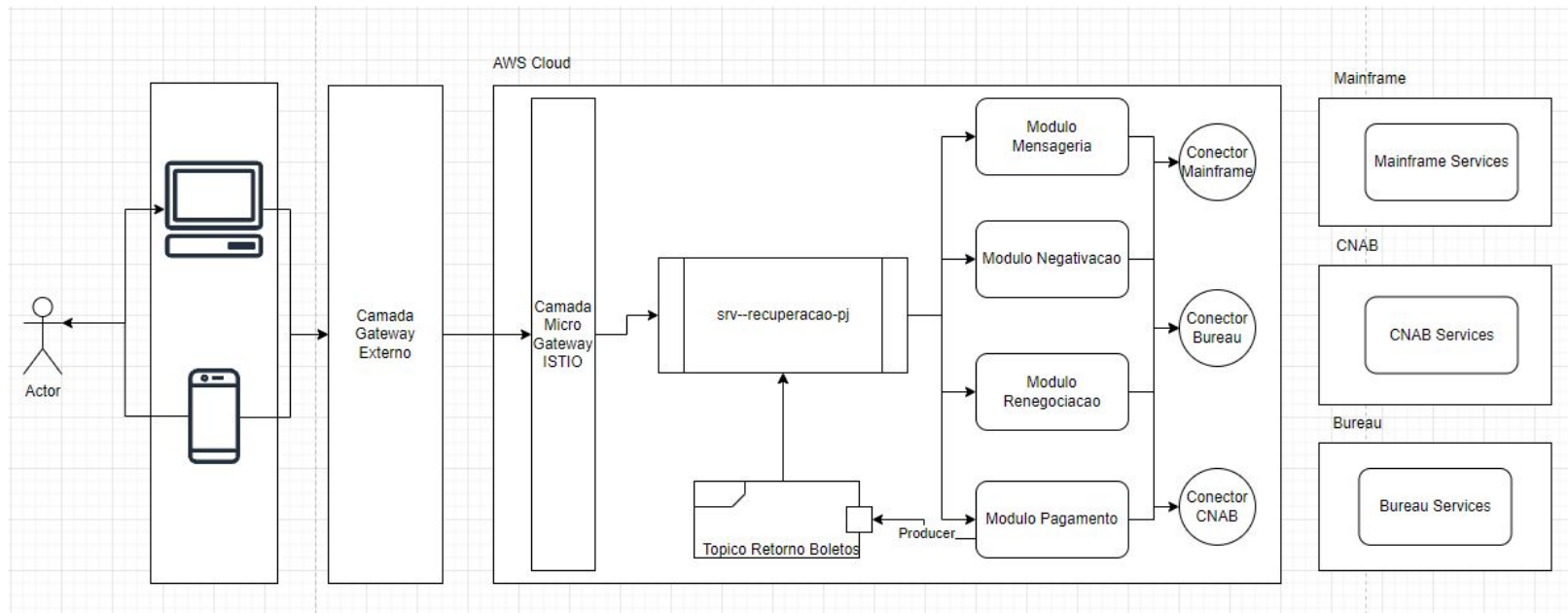
---



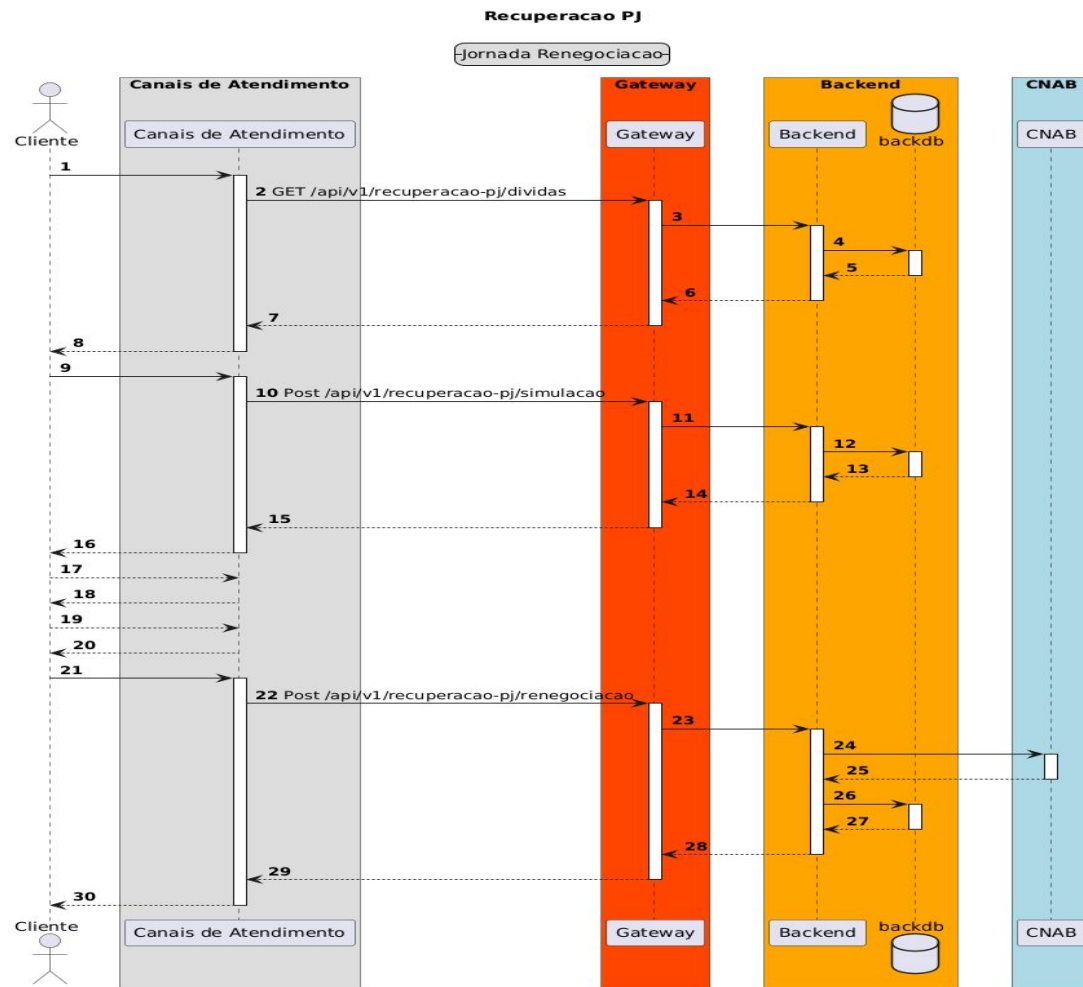
# Introdução e Contexto

- Problema:
  - Necessidade de um sistema eficiente para gerenciar a recuperação de crédito de clientes Pessoa Jurídica.
  - Desafios na integração com sistemas legados e na escalabilidade da solução.
- Objetivo:
  - Desenvolver um sistema robusto e escalável para automatizar e otimizar o processo de recuperação de crédito.
- Solução:
  - Construir uma nova aplicação utilizando se Arquitetura Hexagonal (Ports and Adapters) para realizar a separação entre Domínio e Infraestrutura, facilitar a integração com o legado via API, desacoplar componentes para facilitar Testes Unitários e Testes de Integração e, facilitar a evolução contínua e a transição gradual entre o legado e o novo sem comprometer a operação.

# Arquitetura de Solução



# [Protótipo] Jornada Renegociação



# Tecnologias Utilizadas

- Java 21 - Linguagem madura, performance, comunidade ativa, evolução constante da linguagem.
- Spring Boot 3.x - Desenvolvimento rápido, configuração simplificada, ecossistema rico, microsserviços.
- Gradle 7.6 - Automatização de builds, gerenciamento de dependências, flexibilidade.
- H2 Database - Banco de dados em memória para testes, desenvolvimento rápido.
- Amazon EKS/ECR/SQS - Escalabilidade, resiliência, integração com a infraestrutura da AWS.
- Swagger - Documentação automática da API, testes facilitados.
- JUnit e Mockito - Testes unitários, garantia de qualidade, desenvolvimento orientado a testes.
- JWT - Autenticação e autorização seguras, microsserviços.
- SLF4J e Logback - Logging estruturado, depuração facilitada, monitoramento.
- Micrometer + Prometheus - Coleta de métricas, monitoramento de performance, alertas.
- Spring Cloud Sleuth + OpenTelemetry - Rastreamento distribuído, diagnóstico de problemas, observabilidade.
- Grafana/Kibana - Visualização de dados, dashboards, análise de logs.

# Padrões de Desenvolvimento

- Adapter Pattern: Integração com sistemas legados, adaptação de interfaces.
- Facade Pattern: Simplificação de interfaces complexas, encapsulamento de subsistemas.
- Service Layer Pattern: Separação de lógica de negócio, organização do código.
- Dependency Injection Pattern: Inversão de controle, testes facilitados, baixo acoplamento.
- Use Case Pattern: Organização do código em casos de uso, clareza do fluxo de negócio.
- Command Pattern: Encapsulamento de operações, filas de comandos, histórico de ações.
- Data Transfer Object (DTO): Transferência de dados entre camadas, segurança e performance.
- Repository Pattern: Abstração do acesso a dados, testes facilitados, organização do código.
- Entity Pattern: Modelagem do domínio, persistência de dados, organização do código.

# Motivos das Escolhas

- Java 21 e Spring Boot 3.x - Performance, escalabilidade, comunidade ativa, microsserviços.
- Padrões de desenvolvimento - Organização do código, manutenibilidade, testes, escalabilidade.
- Amazon EKS/ECR/SQS - Infraestrutura robusta, escalabilidade, integração com outros serviços AWS.
- Ferramentas de observabilidade - Monitoramento, diagnóstico de problemas, performance:
  - Micrometer → Coleta métricas de desempenho e uso da aplicação
  - Prometheus → Armazena e consulta essas métricas
  - Grafana/Kibana → Visualiza as métricas coletadas
  - SLF4J + Logback → Geração de logs estruturados e configuráveis (formato JSON)
  - Spring Cloud Sleuth → Propagação de rastreamento distribuído (traceId e spanId)
  - OpenTelemetry → Unifica métricas, logs e traces em um padrão aberto

# Próximos Passos e Considerações Futuras

- Implementação de próximos módulos de forma estratégica;
- Melhorias contínuas na performance e escalabilidade;
- Implementação de novos casos de uso;
- Otimização do monitoramento e observabilidade;
- Possíveis implementações de novas ferramentas de automação de testes;



# Conclusão

Ao longo desta apresentação, exploramos a solução de recuperação de crédito PJ desenvolvida para o Case, destacando os principais aspectos que garantem sua robustez, escalabilidade e eficiência. Recapitulando, vimos:

- **A necessidade de um sistema eficiente:** Enfrentamos o desafio de otimizar a recuperação de crédito PJ, superando as limitações dos sistemas legados e garantindo a escalabilidade da solução.
- **A arquitetura da solução:** Adotamos uma arquitetura em camadas e microsserviços, integrando serviços externos como o Amazon SQS para garantir a resiliência e o desempenho do sistema.
- **As tecnologias utilizadas:** Selecionamos um conjunto de tecnologias modernas e maduras, como Java 21, Spring Boot 3x e serviços da AWS, para garantir a performance, a escalabilidade e a manutenibilidade da solução.
- **Os padrões de desenvolvimento:** Implementamos padrões de desenvolvimento como Adapter, Facade e Repository para garantir a organização do código, a manutenibilidade e a escalabilidade da solução.
- **A importância da observabilidade:** Adotamos ferramentas como Micrometer, Prometheus, Grafana e Kibana para monitorar a performance do sistema, diagnosticar problemas e garantir a qualidade da solução.
- **Os próximos passos:** Planejamos melhorias contínuas na performance e escalabilidade, a implementação de novos casos de uso e a otimização do monitoramento e observabilidade.

Acredito que a solução apresentada representa um avanço significativo na gestão da recuperação de crédito PJ do Itaú, proporcionando maior eficiência, agilidade e controle.

